Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Системы автоматизированного проектирования»

ОТЧЁТ
по лабораторной работе № 2

по дисциплине «Логика и основы алгоритмизации
в инженерных задачах»

Оценка времени выполнения программ

Выполнили студенты группы 25BBB4:
Поликарпов Я., Егоров Е.

Приняли:
Юрова О. В., Деев М. В.

Пенза 2026

Цель и ход работы

Цель работы - изучить оценку времени выполнения программ средствами библиотеки time.h, определить порядок сложности умножения матриц и сравнить время сортировок на массивах с разным исходным порядком.

В ходе работы выполнены два задания: исследовано умножение квадратных матриц для семи заданных размеров и измерено время Shell, Quick и qsort на четырёх типах массива. Умножение имеет кубическую сложность; время сортировок зависит от входных данных.

Задание 1. Умножение матриц

Пункт 1 Определение порядка сложности

В исходной программе из задания исправлены ошибки: добавлен сброс счётчика j перед заполнением каждой строки, присваивание C[i,j] вынесено из цикла по r, в сортировке Шелла проверка j >= 0 выполняется до обращения к элементу массива. Статические массивы 200×200 заменены динамическими, так как размеры достигают 10000.

Матрицы A, B и C хранятся в динамически выделенных одномерных массивах. Элемент C[i,j] вычисляется суммированием N произведений. В таблице показано, как сложность растёт при переходе от отдельной операции к вложенным циклам.

Таблица 1 - Оценка сложности основных операций

Фрагмент алгоритма	Число повторений	Сложность
Одно умножение и прибавление к sum	1	O(1)
Цикл r для одного элемента C[i,j]	N	O(N)
Циклы j и r для одной строки C	N^2	$O(N^2)$
Циклы i, j и r для всей матрицы C	N^3	$O(N^3)$
Заполнение двух исходных матриц	$2N^2$	$O(N^2)$

По правилу сложения сложностей последовательных этапов получаем:

$$T(N) = O(N^2) + O(N^3) = O(N^3).$$

Тройной цикл выполняет ровно N^3 итераций внутреннего тела. В модели с постоянной стоимостью арифметических операций точная асимптотическая оценка времени - $\Theta(N^3)$. Работа распределителя памяти отдельно не моделируется.

Для трёх матриц требуется $3N^2 \cdot$sizeof(int) байт, поэтому сложность по памяти равна $O(N^2)$. Динамическое выделение позволяет хранить большие матрицы вне стека.

Пункт 2 Результаты для всех размеров матриц

Время измеряется функцией clock() из библиотеки time.h. Разность двух показаний переводится в секунды по формуле:

$$T = \frac{finish - start}{CLOCKS_PER_SEC}.$$

Общее время охватывает выделение памяти, заполнение A и B и умножение. Отдельный интервал измеряет только multiply(). Ввод, вывод, контрольная сумма и освобождение памяти находятся за пределами обоих интервалов.

Таблица 2 - Измеренное время умножения матриц

N	Общее время, с	Умножение, с	Контрольная сумма
100	0,002	0,001	497989
200	0,013	0,012	968998
400	0,091	0,088	1968365
1000	1,706	1,689	5042599
2000	15,254	15,173	10238642
4000	170,652	170,402	20566948
10000	3454,601	3453,085	50974129

Матрицы A и B заполнены случайными целыми числами от 1 до 100. Контрольная сумма равна сумме первого и последнего элементов C. Для N = 10000 общее время составило 3454,601 с, то есть около 57 мин 35 с.

Пункт 3 График зависимости времени от размера

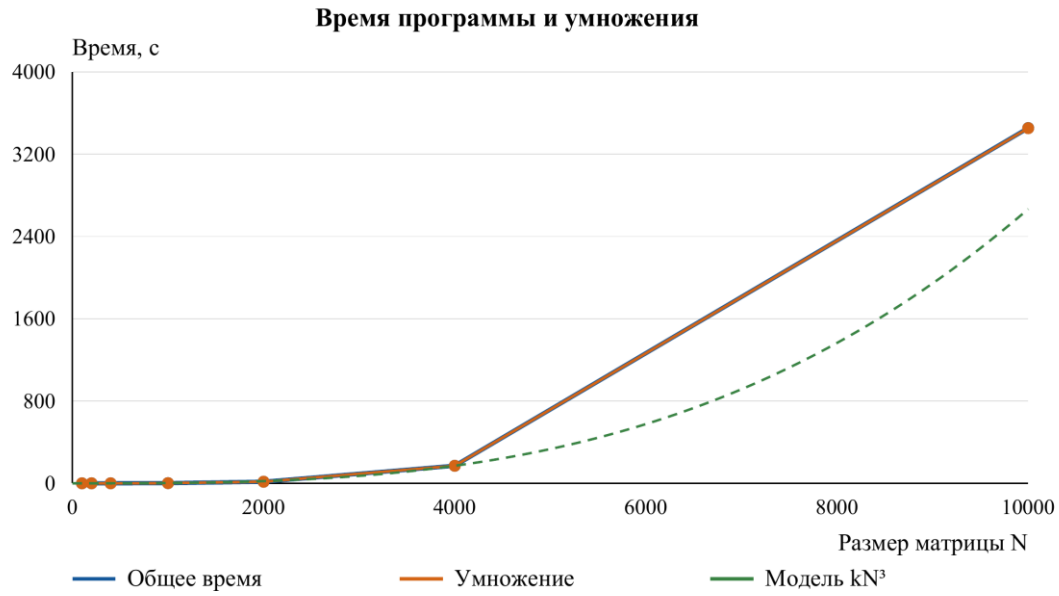


Рисунок 1 - Общее время программы и время умножения

Сравнение с теоретической оценкой

Для сравнения с кубической оценкой используем модель $T(N) = kN^3$. Коэффициент k определён по общему времени при N = 4000:

$$k = \frac{170{,}652}{4000^3} \approx 2{,}66644 \cdot 10^{-9} \text{ с.}$$

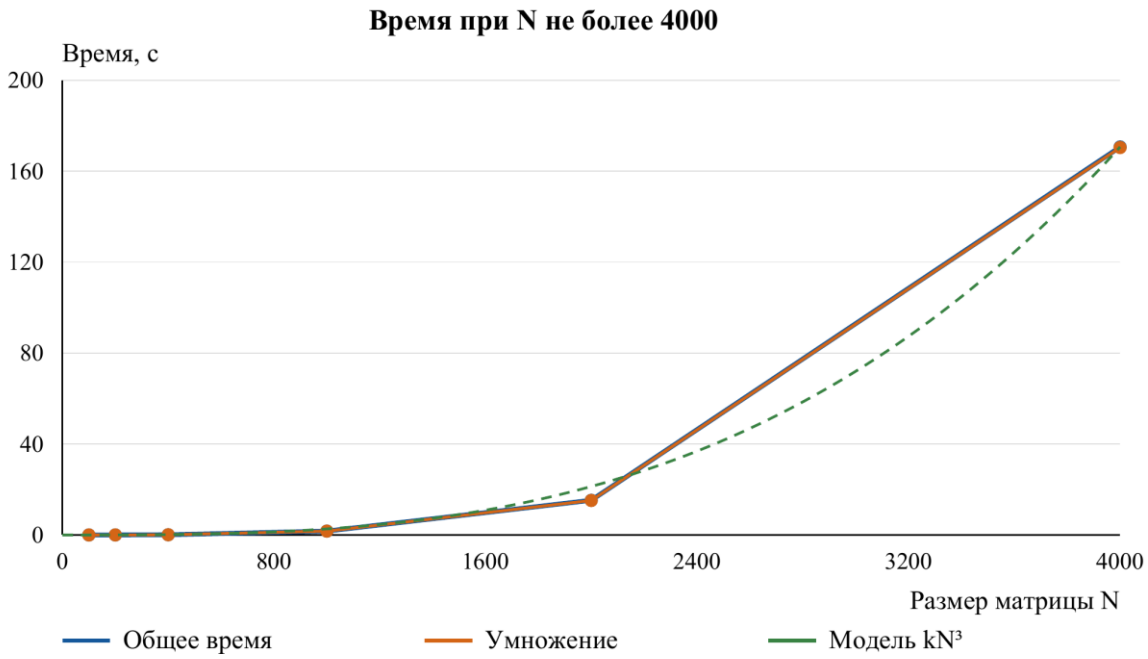


Рисунок 2 - Детализация графика для N не более 4000

Таблица 3 - Общее время и кубическая модель

N	Измерение, с	Модель kN^3, с	Разность, с
100	0,002	0,003	-0,001
200	0,013	0,021	-0,008
400	0,091	0,171	-0,080
1000	1,706	2,666	-0,960
2000	15,254	21,331	-6,077
4000	170,652	170,652	0,000
10000	3454,601	2666,437	788,164

При N = 10000 модель даёт 2666,437 с, а измерение - 3454,601 с. Отклонение составляет около 29,6 %. Точка N = 4000 использована для калибровки, поэтому совпадение в ней не служит независимым подтверждением модели.

Основную долю общего времени занимает умножение. Рост времени согласуется с оценкой $O(N^3)$, но один постоянный коэффициент не описывает все размеры точно. Возможны влияние кэша, пропускной способности памяти и условий запуска; их отдельный вклад в работе не измерялся.

Задание 2. Сравнение сортировок

Использованы сортировка Шелла с шагами {9, 5, 3, 2, 1}, собственная быстрая сортировка Quick и стандартная функция qsort с компаратором целых чисел. Размер массива N = 10000.

Замеры выполнены под Windows в сборке GCC из комплекта CLion без оптимизации (-std=c11 -Wall -Wextra -g). Каждый алгоритм запускается 10 раз на копии одного исходного массива; в таблице указано среднее время. Копирование и проверка неубывающего порядка не входят в замер. Случайный набор создаётся с srand(1), что фиксирует последовательность для используемой стандартной библиотеки.

Таблица 4 - Среднее время сортировки по 10 запускам в секундах

Тип массива	Shell	Quick	qsort
Случайный	0,004300	0,000400	0,000700
Возрастающий	0,000100	0,000200	0,000100
Убывающий	0,007700	0,000100	0,000300
Первая половина возрастает, вторая убывает	0,004000	0,016800	0,000700

Пункты 1 и 2 Случайный и возрастающий массивы

Случайный массив заполнен rand(). Для возрастающего массива A[i] = i при i от 0 до N-1. Шелл на уже упорядоченном массиве выполняет меньше сдвигов и работает быстрее, чем на случайном.

Пункты 3 и 4 Убывающий и смешанный массивы

Убывающий массив задан как A[i] = N-i. В смешанном A[i] = i при i < N/2, иначе A[i] = N-i. На смешанном массиве Quick замедляется: средний по индексу опорный элемент близок к максимуму и создаёт неравномерные разбиения. В худшем случае время Quick равно $O(N^2)$.

Пункт 5 Стандартная функция qsort

Время qsort измерено на тех же четырёх наборах. В этом эксперименте она быстрее Шелла на случайном, убывающем и смешанном массивах. Конкретный внутренний алгоритм qsort зависит от реализации стандартной библиотеки.

Все 120 запусков прошли проверку порядка. Разрешение clock() ограничивает точность коротких замеров; отдельный запуск может дать ноль. Усреднение не повышает разрешение самого таймера, поэтому различия очень малых значений нельзя считать устойчивым преимуществом алгоритма.

Псевдокод алгоритмов и измерений

Умножение матриц

```
УмножениеМатриц(N):
  total_start := clock()
  выделить A, B, C размером N×N; проверить выделение
  заполнить A и B случайными числами от 1 до 100
  mult_start := clock()
  для i := 0 .. N-1:
    для j := 0 .. N-1:
      sum := 0
      для r := 0 .. N-1:
        sum := sum + A[i,r] * B[r,j]
      C[i,j] := sum
  mult_end := clock(); total_end := clock()
  вычислить checksum; вывести времена и checksum
  освободить A, B, C
```

Сортировка Шелла

```
ShellSort(A, N):
  для gap из [9, 5, 3, 2, 1]:
    для i := gap .. N-1:
      x := A[i]; j := i
      пока j >= gap И x < A[j-gap]:
        A[j] := A[j-gap]; j := j-gap
      A[j] := x
```

Измерение среднего времени сортировки

```
Measure(sort, source, work, N):
  elapsed := 0
  повторить 10 раз:
    скопировать source в work
    begin := clock(); sort(work, N); end := clock()
    проверить неубывающий порядок work
    при ошибке завершить программу
    elapsed := elapsed + (end-begin) / CLOCKS_PER_SEC
  вернуть elapsed / 10
```

Псевдокод QuickSort

```
QuickSort(A, left, right):
  пока left < right:
    i := left; j := right
    pivot := A[left + (right-left) / 2]
    повторять:
      пока A[i] < pivot И i < right: i := i+1
      пока pivot < A[j] И j > left: j := j-1
      если i <= j:
        обменять A[i], A[j]; i := i+1; j := j-1
    пока i <= j
    если j-left < right-i:
      если left < j: QuickSort(A, left, j)
      left := i
    иначе:
      если i < right: QuickSort(A, i, right)
      right := j
```

Рекурсивно сортируется только меньшая часть, а большая обрабатывается циклом. Это ограничивает глубину стека величиной O(log N) и предотвращает его переполнение. Неудачный выбор опорного элемента по-прежнему может приводить к квадратичному времени.

Вывод

В ходе работы мы освоили измерение времени программ с помощью time.h. Для умножения матриц установили сложность $O(N^3)$ по времени и $O(N^2)$ по памяти, измерили время для всех семи заданных размеров и сравнили его с кубической моделью. При N = 10000 общее время составило 3454,601 с, из них 3453,085 с заняло умножение.

Сравнение Shell, Quick и qsort на четырёх типах массива показало зависимость времени от исходного порядка. Шелл быстрее обрабатывает возрастающий массив, а Quick замедляется на смешанном из-за неравномерных разбиений. Для каждого сочетания алгоритма и входного набора получили среднее по 10 запускам и проверили неубывающий порядок результата.